

# Breakout Lab 3.1

---

Parallel Agent Coordination

30 minutes | Groups of two

# Mission

Complete email and Short Message Service (SMS) workers in isolation, then integrate their evidence.

## COLLABORATION

### Format

One driver per session; each person reviews the other handoff.

## ALLOWED STACK

### Tools

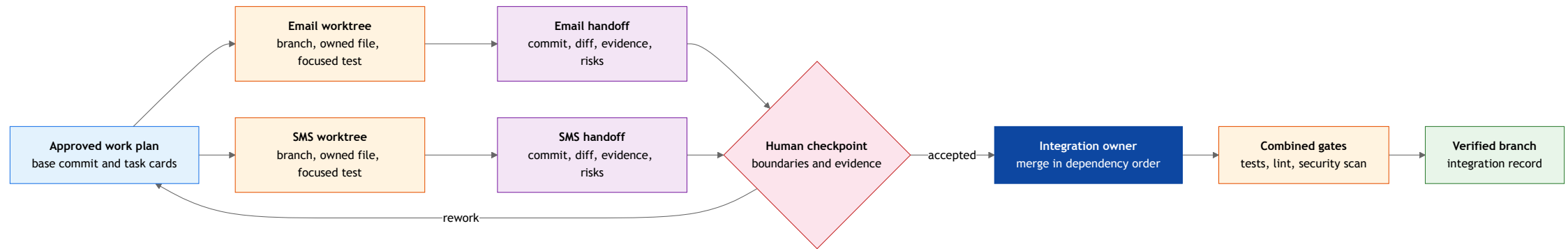
Claude Code, Git worktrees, Python, and `unittest`.

## REQUIRED EVIDENCE

### Output

Commits, handoffs, ordered merges, checks, and one approval point.

# Parallel Work, Controlled Integration



# Interactive: MVP Test Factory Workflow

See a real example of this pattern: a shared setup with two human approvals, then three independent test-writing paths (API, UI, performance) that converge on one fixed-order gate sequence before human review.

Open `mvp-test-factory-workflow.html` from `claude-helper/docs/interactive/` to compare the setup path against the parallel-authoring path.

# Create a Fresh Repository

From `day1/lab3_1_parallel-agent-coordination/` in the companion repository:

```
python3 start/setup_lab.py lab-workspace  
cd lab-workspace  
python3 -m unittest
```

Expect failures only from the two incomplete channel renderers.

# 30-Minute Run of Show

| Time  | Checkpoint                                       |
|-------|--------------------------------------------------|
| 0-3   | Create the repository and inspect task cards.    |
| 3-6   | Assign worktrees, branches, and file boundaries. |
| 6-15  | Run two bounded sessions and collect handoffs.   |
| 15-20 | Cross-review diffs, commits, checks, and schema. |
| 20-25 | Approve, merge in order, and run combined gates. |
| 25-30 | Resolve the fixture, verify, and share evidence. |

# Open Two Isolated Sessions

```
git worktree add ../work-email -b agent/email  
git worktree add ../work-sms -b agent/sms  
git worktree list
```

Use the exact email and SMS prompts in the README. Each session changes only its channel file and handoff.

# Review Before the Second Merge

For both branches, inspect the diff, commit, focused result, handoff, and unchanged schema.

```
git merge --no-edit agent/email  
python3 -m unittest tests.test_email tests.test_security.ChannelSecurityTest.test_email_escapes_untrusted_html
```

A human must approve both handoffs before `git merge --no-edit agent/sms`.

# Practice the Shared-File Conflict

Create `conflict-practice`, merge both fixture branches, and preserve:

```
CHANNEL_ORDER = ("email", "sms")
```

Run the full suite after resolving. Do not merge the practice branch into `main`.

If the group is behind, preserve the core handoff and integration evidence; debrief the prepared conflict with the instructor.

# Verify and Share Evidence

Back on `main`, complete `integration-record.json`, then run:

```
python3 -m unittest  
python3 verify_record.py
```

Show one handoff and the combined test output. Explain one enforced boundary and why a clean merge is not proof of compatibility.

# Definition of Done

If blocked, use the exact failure, smallest owned file, task card, peer hint, then `solution/`.

## LINEAGE

Both worker commits descend from the approved `lab-base`.

## CHECKS

Focused, full, and security checks pass.

## INTEGRATION EVIDENCE

Approval precedes the second merge;  
`verify_record.py` accepts the record.

## CONFLICT PRACTICE

The resolved fixture preserves the approved channel order.

# Return for the Course Wrap-Up

---

Bring one handoff, the combined check, and one integration decision.